

CHAPTER 1:

INTRODUCTION TO

EMBEDDED SYSTEMS

LEARNING OBJECTIVES

In this chapter we will learn:

- What an embedded system is
- Why embed a computer
- What attributes embedded systems need to provide
- What constraints do embedded systems have

WHAT IS AN EMBEDDED SYSTEM?

An embedded system refers to a digital computer that is... embedded into another system! Think about all the devices you interact with on a daily basis and which ones have been “digitized” by the inclusion of a small computer integrated into them to provide some sort of control or automation. Did you make coffee this morning with a coffee maker that automatically poured the perfect amount of coffee for your cup? Did you drive to work in a car that had a “push-to-start” button? Did you wash your clothes this week with a washing machine that knew how long to run each wash cycle based on the load size? These are all examples of systems that are capable of automating many tasks due to their embedded computer!

On the other hand, what is not an embedded system? An embedded system is integrated into another device to provide some sort of control or automation. A computer that we use for general computing purposes, such as a laptop computer, would not be considered an embedded system.

A great rule of thumb for determining whether a computer is an embedded system or not, is to evaluate what the system is being used for. If the device contains a computer, but is purchased or sold not for computing purposes, then it is an embedded system. If the device is purchased or sold because it is a computer, then it is not an embedded system. For example, if you bought a coffee maker that has an embedded computer, do you care how much memory it has or how fast the processor is?

EMBEDDED SYSTEMS AND MICROCONTROLLERS

We can’t talk about embedded systems without discussing microcontrollers. A **microcontroller** is a device that contains an entire computer on a single chip, making it very easy to integrate with other systems. How are we able to fit an entire computer onto a single chip? Given the perspective of viewing a computer as something

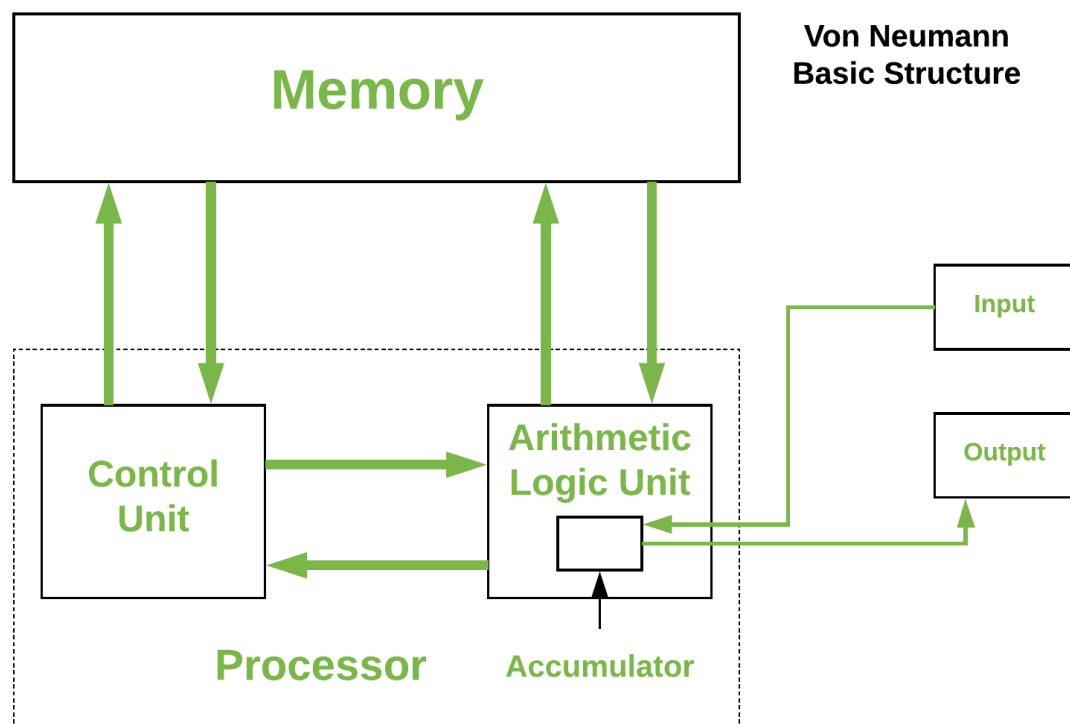
powerful like a laptop or desktop, this concept might feel a little far-fetched, but keep in mind we don't need our coffee maker to run "photoshop".

Because of the limited needs of the application, the computer might not need very many resources, allowing it to be stripped of any excess memory and processing power and shrunk down to a very small size, perfect for integration. Many microcontrollers will have memory on the scale of Megabytes while your laptop is easily on the Gigabyte scale. Microcontrollers come in a variety of different specifications - and it is up to the engineer to select the one most appropriate for the system in which its being embedded, based on processing power, memory, and input/output demands.

MICROCONTROLLERS VS. MICROPROCESSORS

The term "microcontroller" may sound familiar to you, due to its similarity to the term "microprocessor". A microprocessor is the "brain" of a computer, such as the AMD or Intel processors found in a laptop or desktop computer. Microprocessors are often quite powerful, especially when contrasted with the computational power of most microcontrollers, but they themselves are not an entire computer.

To understand what constitutes a computer, let's take a look at the five basic components of a Von-Neuman computer, which is the architecture of most modern computers:



In this figure, we can identify five major components:

- Memory
- Arithmetic Logic Unit
- Control Unit
- Input
- Output

The memory is responsible for holding the computer's information, such as the instructions comprising the programs and any data currently in use. The arithmetic logic unit (ALU) is capable of performing computations on the data, such as addition, subtraction, etc. The control unit reads instructions and directs the ALU to perform the appropriate action. The input collects data from the outside, such as through a keyboard or mouse, and the output can display information, such as a monitor, or perhaps even manipulate something, like printing a page on a printer.

The ALU and control unit are often bundled together and called the "processor". These two components are what comprise the microprocessor as well! As we can see here, the microprocessor is only two out of five components necessary for making an entire computer. The microcontroller actually contains all five of these components, but often in a more limited capacity than your laptop.

WHY EMBED A COMPUTER?

Embedding a computer into another system can bring many advantages. Since computers can be programmed to create a variety of behaviors, they can replace the need for engineers to design a custom circuit to perform specific tasks. Say an engineer is tasked with creating a circuit that will blink a warning light on a car's dashboard with a one second interval. This would usually require a custom circuit with a specific timer and clock frequency, but rather than design this circuit, the computer can be integrated and programmed to blink this light. Not only would it be capable of handling this task, but many others at the same time. This is a simple example, but microcontrollers can also imitate the behavior of much more complex circuits as well.

Embedding computer to avoid having engineers create custom circuits cuts down on a product's non-recurring engineering (NRE) cost. This is the cost of paying an engineer to create a new custom design. The engineer will still have to program the microcontroller, but this is generally easier than building an entire circuit.

Microcontrollers can also help reduce the cost of a design due to the fact that they are a generic component. The same microcontroller can be integrated into many different systems and be programmed for very different tasks. Microcontrollers will be manufactured on a scale that makes them extremely affordable, often costing less than 25 cent per chip. Even though the microcontroller's design may be far more complex than a custom circuit for a specific task, its mass manufacturing still makes it the more cost-effective option.

AN EXAMPLE OF EMBEDDING COMPUTERS

As an example, let's examine how embedded systems have affected automobiles. A typical modern car has dozens of microcontrollers embedded within it. Let's see why.

Computer control of automobile engines lowers pollution and increases fuel efficiency, reducing operating costs. Burning gasoline with air in spark ignition engines is a tricky business if we want to maximize efficiency, yet minimize pollution. The main factor affecting emissions is the ratio of air mass to fuel mass. The ideal ratio is 14.7 to 1, and the catalytic converter is designed to operate most efficiently at this ratio. If there is too little air (a rich mix), then excessive carbon monoxide (CO) and hydrocarbons (HC) will be produced. If there is too much air (a lean mix), then large amounts of oxides of nitrogen (called NO_x) will be created. So we would like for each fuel injector to add just the right amount of fuel. This depends on the mass of the air inside the cylinder, which depends on factors such as air temperature and air pressure. These in turn depend on altitude and weather, as well as whether the engine is warmed up or not.

Another factor is the timing of the sparkplug firing. If it fires early, then there is more time for combustion within the cylinder before the exhaust valve opens. This raises the average temperature within the cylinder and changes the combustion process, affecting CO, HC, and NO_x concentrations.

It would be quite impractical to design a mechanical control system to consider all of these factors and squirt the fuel injectors at just the right instant for the right amount of time. Thankfully, an inexpensive microcontroller is quite effective at these kinds of calculations and control.

LIMITATIONS OF EMBEDDED SYSTEMS

While we have observed many benefits that can come from embedded systems, such as cost reductions and automation, there are also many limitations that must be considered when designing for them. It was mentioned earlier that embedded systems are not as powerful as the microprocessors found in your laptop computer, and therefore their computing power and other resources are limited. This must be kept in mind when selecting the embedded system appropriate for the application.

If limited resources are such a concern, why not use a microcontroller with more memory and processing power than what's required for the application? There are two reasons for this: cost and power consumption. Cost is the more obvious of the two – more resources equals more hardware equals higher cost. This cost may only increase the microcontroller's cost by a few cents, but when a the product that is utilizing the microcontroller is being manufactured on the scale of thousands to millions of units, these pennies begin to add up. Power consumption is another major concern. More memory means more hardware components that require power, and faster processors have a higher clock frequency, which means more transistors switching, which means higher power consumption. Embedded systems often find themselves at home in battery-operated mobile devices, therefore keeping the power consumption low is of utmost importance to ensure battery life is elongated as much as possible. Have you ever purchased one device, such as a smartphone or laptop, over another because one had a longer battery life?

Embedded systems are also sometimes limited by their input and output capabilities. Most embedded systems come with some features that allow them to produce digital signals, read analog voltages, and communicate with other devices using serial protocols. These features can vary from microcontroller to microcontroller, so one must be selected that contains the necessary interfacing hardware, cost, and computational resources. It

is the embedded systems engineer's responsibility to select the correct microcontroller based on the project's demands for these criteria.

DEVELOPING FOR EMBEDDED SYSTEMS

When developing for an embedded system, the design must first go through a prototyping phase to verify functionality. This prototyping phase usually involves breadboarding the circuit that will integrate the selected microcontroller, and the microcontroller will often be interfaced through a **development board**.

Development boards are prototyping boards that contain a microcontroller and onboard tools and components that aid in development and testing. These components often include the following:

- Voltage regulation
- Programmer circuit
- Debugger
- Breakout header pins
- Push buttons and LEDs

These components often appear on development boards because they are commonly needed by programmers to power the board, upload code, and test inputs and outputs.

Let's look at an example development board, the MSP-EXP430FR2355. This board has a lot going on, but what is most important is the chip at the center of the board, the MSP430FR2355. This is the microcontroller, and all the other components here are to help with programming and testing of this chip.

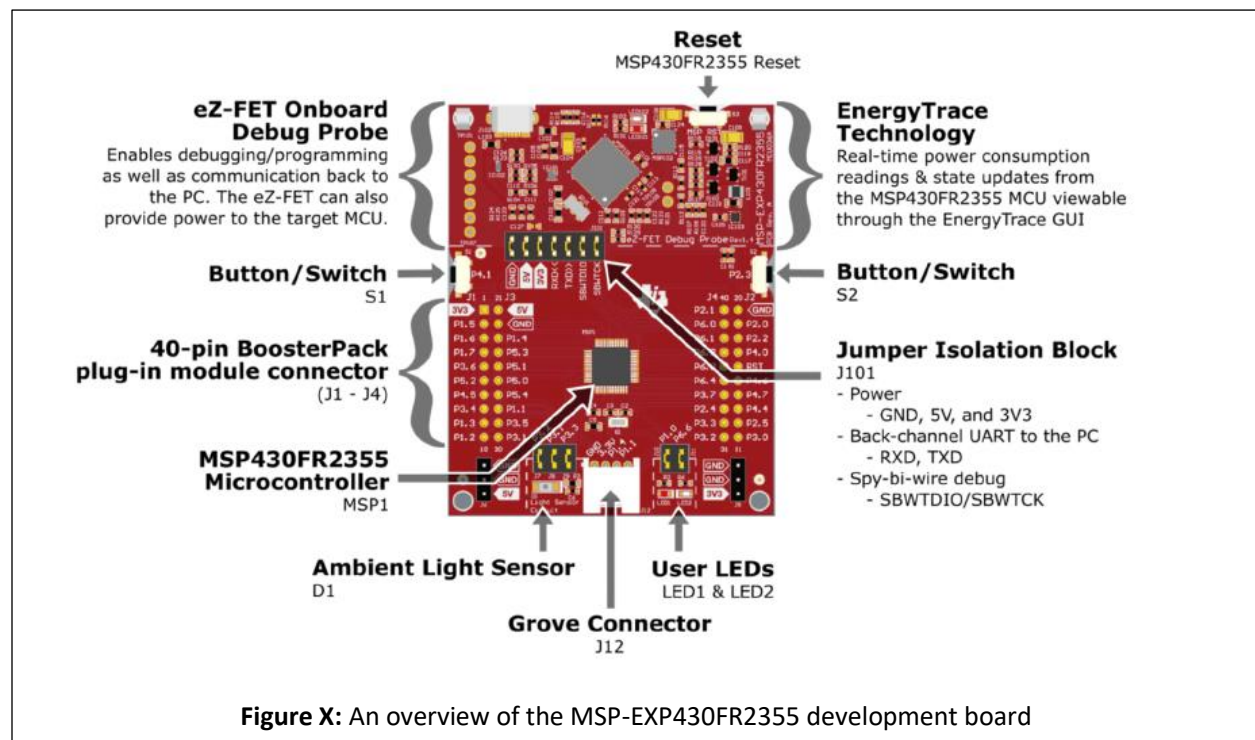


Figure X: An overview of the MSP-EXP430FR2355 development board

The top half of the board contains the “eZ-FET” debugger circuitry. A debugger allows us to meticulously examine our microcontroller’s code behavior by setting breakpoints and stepping through the code line by line, while simultaneously being able to check memory contents. If this doesn’t make much sense right now, don’t worry, we will be elaborating on this again later in the book. This portion of the board also provides power regulation, which provides power to the microcontroller at a specific voltage. This part of the board also provides a virtual serial port connection from the microcontroller to your laptop or desktop computer, allowing communication between the two.

Separating the top “half” from the bottom, we see a “jumper isolation block”. These jumpers bridge electrical connections between the debugger circuit and the microcontroller. These bridged connections allow power and communications to reach the lower part. However, if we were planning on someday building a custom circuit using the MSP430G2355, we wouldn’t be including a debugger/programmer, since we wouldn’t expect the person purchasing the embedded product, the “end user”, to reprogram the device. These jumpers allow us to disconnect the microcontroller and test the chip without the support of that circuit, emulating what it might behave like before we create a custom board.

On the lower half of the board, beyond the jumper isolation block, we have the microcontroller itself, along with some extra connected components. Arguably, the most important addition here, is the inclusion of “breakout” pins. Breakout pins are pins that are large and spaced out, designed to be easily connected to jumper wires for interfacing with a prototyping circuit. The pins on a typical microcontroller package are much too small to attempt to plug or solder wires directly to, so the development board contains traces (wires embedded into the board) that connect these tiny package pins to the breakout pins.

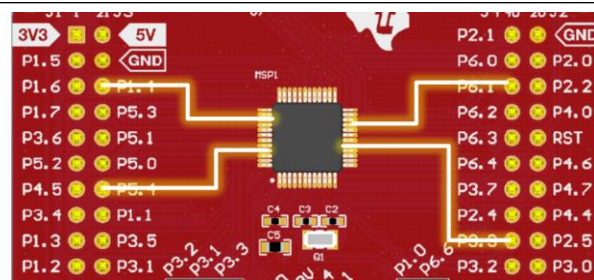


Figure X: A few examples of what traces connect the breakout pins to the tiny pins on the microcontroller.

In addition to the breakout pins, development board very commonly include components such as buttons and LEDs, as they are very often used in prototyping scenarios for easy getting user input via a button press or indicating the status of something through illuminating an LED.

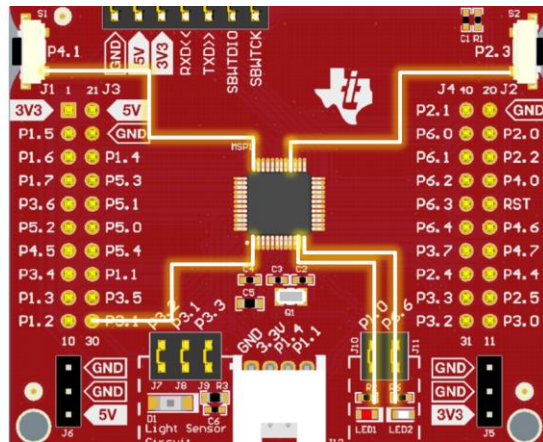


Figure X: Traces connecting the pins of the microcontroller to the various buttons and LEDs on the development board.

At the very bottom of the board is a white connector, which is part of a pinout standard developed by Seeed Studio called the “Grove”. The Grove interface is used to easily connect various LEDs, buttons, sensors, speakers, etc., easily without the need for using a breadboard or soldering.

RECAP

An embedded system is an application-specific computer system which is built into a larger system or device. Using a computer system enables improved performance, more functions and features, lower cost, and greater dependability. With embedded computer systems, manufacturers can add sophisticated control and monitoring to devices and other systems, while leveraging the low-cost microcontrollers running custom software.

REFERENCES